WEEK 06 · BLOCK 6B

Block 6b — Interactive to-do list

Goal: build a working to-do app with plain DOM + events — add items via a form (no reload), mark them done, and delete them. This app uses **everything** from Block 6.

Time: ~45 minutes · **You need:** a browser and a terminal (Node from Block 0)

Setup

```
cd exercises/week06/6b/starter
npm install
npm start       # http://localhost:3000
```

Open http://localhost:3000. The form and styling are done; you write the behaviour in `app.js`. Reload after each change.

Task 1 — Add an item (no reload)

Listen for the form's `submit` event:

- `e.preventDefault()` — stop the page reloading *(Block 1)*
- read `input.value`, `.trim()` it; if it's **empty**, `return` (ignore it)
- call `addItem(text)`
- clear the field: `input.value = ""`

Task 2 — Build the item

Finish `addItem(text)`. Create an `<li>` that contains:

- the todo **text** (use `textContent`, **not** `innerHTML` — user input!)
- a delete **button**: `<button class="delete">×</button>`

Then `append` the `<li>` to `list`. Adding a few todos should now stack them up.

> Tip: `createElement` a `<span>` for the text and a `<button>` for delete, then `li.append(span, button)`.

Task 3 — Toggle done & delete (event delegation)

Add **one** `click` listener to `list` (not to each item). Inside it:

- if the click hit a `.delete` button → remove that `<li>` (`e.target.closest("li").remove()`)

- otherwise → **toggle** the `done` class on the clicked item (`e.target.closest("li").classList.toggle("done")`)

Because the listener is on the parent, it works for items added **later** too — that's the whole point of delegation.

Requirements checklist

- [ ] Submitting the form adds an item and does **not** reload the page
- [ ] Empty / whitespace-only input is ignored; the field clears after adding
- [ ] Each item has a working **delete** button
- [ ] Clicking an item toggles the struck-through **done** style
- [ ] Add/delete/toggle all keep working for items created after page load

Stretch goals (optional)

- Show a live **count** ("3 items, 1 done") above the list, updated on every change.
- Refactor to the **state** → **render** shape from the lecture: keep a `todos` array, write a `render()` that redraws the list, and call it after every change. (This is exactly how the SPA in Block 8 works.)
- Persist the list in `localStorage` so it survives a reload.